

Rozwiązanie: Serwer Odcinków (Java)

Poniżej znajduje się kompletne rozwiązanie zadania z kolokwium [cite: 23, 24]. Program implementuje serwer sieciowy [cite: 25] oraz interfejs graficzny JavaFX zgodnie z wymogami (okno 500x500 [cite: 26], wielu klientów [cite: 27], rysowanie linii [cite: 32, 35, 36], zmiana kolorów [cite: 29, 30, 39, 40, 41] oraz przesuwanie widoku strzałkami [cite: 42, 43]).

Kod źródłowy (LineServer.java)

```
import javafx.application.Application;
import javafx.application.Platform;
import javafx.scene.Scene;
import javafx.scene.canvas.Canvas;
import javafx.scene.canvas.GraphicsContext;
import javafx.scene.control.Label;
import javafx.scene.layout.BorderPane;
import javafx.scene.paint.Color;
import javafx.stage.Stage;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.ArrayList;
import java.util.List;

public class LineServer extends Application {
    private double offsetX = 0;
    private double offsetY = 0;
    private final List<LineSegment> segments = new ArrayList<>();
    private Canvas canvas;
    private Label statusLabel;

    private static class LineSegment {
        double x1, y1, x2, y2;
        Color color;
        LineSegment(double x1, double y1, double x2, double y2, Color color) {
            this.x1 = x1; this.y1 = y1; this.x2 = x2; this.y2 = y2; this.color = color;
        }
    }

    @Override
    public void start(Stage primaryStage) {
        // Inicjalizacja okna 500x500 zgodnie z wymogiem
        canvas = new Canvas(500, 500);
        statusLabel = new Label("Wektor przesunięcia: (0.0, 0.0)");
        statusLabel.setStyle("-fx-padding: 5px; -fx-font-weight: bold; -fx-background-color: white;");

        BorderPane root = new BorderPane();
        root.setCenter(canvas);
        root.setTop(statusLabel);

        // Białe tło
        Scene scene = new Scene(root, 500, 500, Color.WHITE);

        // Obsługa klawiatury dla przesuwania układu (co 10 pikseli)
        scene.setOnKeyPressed(event -> {
            switch (event.getCode()) {
                case LEFT: offsetX -= 10; break;
                case RIGHT: offsetX += 10; break;
                case UP: offsetY -= 10; break;
                case DOWN: offsetY += 10; break;
            }
        });
    }
}
```

```

        default: return;
    }
    updateView();
});

primaryStage.setTitle("Serwer Odcinków");
primaryStage.setScene(scene);
primaryStage.setResizable(false);
primaryStage.show();

// Uruchomienie wątku serwera sieciowego obok GUI
Thread serverThread = new Thread(this::runServer);
serverThread.setDaemon(true);
serverThread.start();

updateView();
}

private void updateView() {
    Platform.runLater(() -> {
        statusLabel.setText(String.format("Wektor przesunięcia: (%.1f, %.1f)",
offsetX, offsetY));
        GraphicsContext gc = canvas.getGraphicsContext2D();

        // Czyszczenie tła na białe
        gc.setFill(Color.WHITE);
        gc.fillRect(0, 0, canvas.getWidth(), canvas.getHeight());

        synchronized (segments) {
            for (LineSegment seg : segments) {
                gc.setStroke(seg.color);
                gc.setLineWidth(2.0);
                // Odcinki rysowane w odwrotnym przesunięciu względem układu
                // (oś odciętych rośnie w prawo, rzędnych w dół)
                gc.strokeLine(seg.x1 - offsetX, seg.y1 - offsetY, seg.x2 - offsetX,
seg.y2 - offsetY);
            }
        }
    });
}

private void runServer() {
    try (ServerSocket serverSocket = new ServerSocket(12345)) {
        while (true) {
            Socket clientSocket = serverSocket.accept();
            // Podłączenie dowolnej liczby klientów - nowy wątek dla każdego klienta
            Thread clientThread = new Thread(() -> handleClient(clientSocket));
            clientThread.setDaemon(true);
            clientThread.start();
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

private void handleClient(Socket socket) {
    // Początkowy kolor przed wyborem to czarny
    Color currentColor = Color.BLACK;
    try (BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream())))
    {
        String line;
        while ((line = in.readLine()) != null) {
            line = line.trim();
            if (line.isEmpty()) continue;

            // Rodzaj 1: Kolor - pojedyncza sześciocyfrowa liczba szesnastkowa
            if (line.matches("^#?[0-9a-fA-F]{6}$")) {
                if (!line.startsWith("#")) {
                    line = "#" + line;
                }
            }
        }
    }
}

```

```

        currentColor = Color.web(line);
    } else {
        // Rodzaj 2: Współrzędne - 4 liczby zmiennoprzecinkowe
        String[] parts = line.split("\\s+");
        if (parts.length == 4) {
            try {
                double x1 = Double.parseDouble(parts[0]);
                double y1 = Double.parseDouble(parts[1]);
                double x2 = Double.parseDouble(parts[2]);
                double y2 = Double.parseDouble(parts[3]);

                synchronized (segments) {
                    segments.add(new LineSegment(x1, y1, x2, y2,
currentColor));
                }
                updateView();
            } catch (NumberFormatException e) {
                // Zakładamy poprawność wiadomości, ignorujemy wadliwe
            }
        }
    }
} catch (Exception e) {
    // Serwer nie wysyła wiadomości zwrotnej klientom, po prostu nasłuchuje do
końca
    e.printStackTrace();
}

public static void main(String[] args) {
    launch(args);
}
}

```

Instrukcja uruchomienia

- Program należy skompilować, upewniając się, że podpięte są biblioteki **JavaFX**. Tworzy on jednocześnie pojedynczy projekt zawierający zarówno klienta GUI, jak i serwer [cite: 24, 25].
- Domyślny port w kodzie to **12345**.
- Do testowania komunikacji w terminalu możesz wykorzystać program `telnet` lub `nc` (netcat), wpisując: `nc localhost 12345`.
- Przykładowe wejścia od klienta po podłączeniu:
 - `FF0000` lub `#FF0000` (zmieni to na stałe kolor z czarnego na czerwony dla przyszłych wprowadzonych odcinków przez tego klienta [cite: 29, 41])
 - `10.5 10.5 250.0 250.0` (rysuje na stałe nowy odcinek [cite: 31, 35, 36])